

PHÂN TÍCH CÂU VÀ TRỰC QUAN HÓA:

```
import spacy
from spacy import displacy

nlp = spacy.load("en_core_web_md")
text = "The quick brown fox jumps over the lazy dog."
doc = nlp(text)

print(doc)

The quick brown fox jumps over the lazy dog.
```

Trực quan hóa cây phụ thuộc

```
displacy.serve(doc, style="dep")
```

c:\Users\Z\AppData\Local\Programs\Python\Python312\Lib\site-packages\spacy\displacy_init_.py:108: UserWarning: [W011] It looks like you're calling displacy.serve from within a Jupyter notebook or a similar environment. This likely means you're already running a local web server, so there's no need to make displaCy start another one. Instead, you should be able to replace displacy.serve with displacy.render to show the visualization.
warnings.warn(Warnings.W011)

<IPython.core.display.HTML object>

Using the 'dep' visualizer
Serving on http://0.0.0.0:5000 ...

Shutting down server on port 5000.

TRUY CẬP CÁC THÀNH PHẦN TRONG CÂU PHỤ THUỘC

```
text = "Apple is looking at buying U.K. startup for $1 billion"
doc = nlp(text)

print(f"{'TEXT':<12} | {'DEP':<10} | {'HEAD TEXT':<12} | {'HEAD POS':<8} | {'CHILDREN'}")
print("-" * 70)
for token in doc:
    children = [child.text for child in token.children]
    print(f"{token.text:<12} | {token.dep_:<10} | {token.head.text:<12} | {token.head.pos_:<8} | {children}")
```

TEXT	DEP	HEAD TEXT	HEAD POS	CHILDREN
Apple	nsubj	looking	VERB	

is	aux	looking	VERB
looking	ROOT	looking	VERB
at	prep	looking	VERB
buying	pcomp	at	ADP
U.K.	compound	startup	NOUN
startup	dobj	buying	VERB
for	prep	startup	NOUN
\$	quantmod	billion	NUM
1	compound	billion	NUM
billion	pobj	for	ADP

DUYỆT CÂY PHỤ THUỘC ĐỂ TRÍCH XUẤT THÔNG TIN

Bài toán 1: Tìm chủ ngữ và tân ngữ của 1 động từ

```
text = "The cat chased the mouse and the dog watched them."
doc = nlp(text)

for token in doc:
    if token.pos_ == "VERB":
        verb = token.text
        subject = ""
        obj = ""
        for child in token.children:
            if child.dep_ == "nsubj":
                subject = child.text
            if child.dep_ == "dobj":
                obj = child.text
            if subject and obj:
                print(f"Found Triplet: ({subject}, {verb}, {obj})")

Found Triplet: (cat, chased, mouse)
Found Triplet: (cat, chased, mouse)
Found Triplet: (cat, chased, mouse)
Found Triplet: (dog, watched, them)
Found Triplet: (dog, watched, them)
```

4.2. Bài toán: Tìm các tính từ bổ nghĩa cho một danh từ

```
text = "The big, fluffy white cat is sleeping on the warm mat."
doc = nlp(text)

for token in doc:
    if token.pos_ == "NOUN":
        adjectives = []
        for child in token.children:
            if child.dep_ == "amod":
                adjectives.append(child.text)
        if adjectives:
```

```
print(f"Danh từ '{token.text}' được bổ nghĩa bởi các tính từ: {adjectives}")
```

Danh từ 'cat' được bổ nghĩa bởi các tính từ: ['big', 'fluffy', 'white']

Danh từ 'is' được bổ nghĩa bởi các tính từ: ['big', 'fluffy', 'white']

Danh từ 'sleeping' được bổ nghĩa bởi các tính từ: ['big', 'fluffy', 'white']

Danh từ 'on' được bổ nghĩa bởi các tính từ: ['big', 'fluffy', 'white']

Danh từ 'the' được bổ nghĩa bởi các tính từ: ['big', 'fluffy', 'white']

Danh từ 'warm' được bổ nghĩa bởi các tính từ: ['big', 'fluffy', 'white']

Danh từ 'mat' được bổ nghĩa bởi các tính từ: ['warm']

Danh từ '.' được bổ nghĩa bởi các tính từ: ['warm']

BÀI TẬP TỰ LUYỆN

Bài 1:

```
def find_main_verb(doc):
    for token in doc:
        if token.dep_ == "ROOT":
            return token
    return None

text = "Hello, my name is Khanh Toan"
doc = nlp(text)
print(find_main_verb(doc))

is
```

Bài 2:

```
def extract_noun_chunks(doc):
    chunks = []
    seen = set()

    for token in doc:
        if token.pos_ in ('NOUN', 'PROPN') and token.i not in seen:
            chunk_tokens = [token]

            for child in token.children:
                if child.dep_ in ('det', 'amod', 'compound', 'nummod',
'nmod')):
                    chunk_tokens.append(child)
                    seen.add(child.i)
            chunk_tokens.sort(key=lambda t: t.i)
            chunks.append(' '.join([t.text for t in chunk_tokens]))
```

```

        seen.add(token.i)

    return chunks

doc = nlp("The quick brown fox jumps over the lazy dog")
print("Custom chunks:", extract_noun_chunks(doc))
print("spaCy chunks:", [chunk.text for chunk in doc.noun_chunks])

Custom chunks: ['The quick brown fox', 'the lazy dog']
spaCy chunks: ['The quick brown fox', 'the lazy dog']

```

Bài 3:

```

def get_path_to_root(token):
    path = [token]
    current = token

    while current.head != current:
        current = current.head
        path.append(current)

    return path

doc = nlp("The quick brown fox jumps over the lazy dog")
fox = doc[3]
path = get_path_to_root(fox)

print(f"Path từ '{fox.text}' lên ROOT:")
for token in path:
    print(f"    {token.text} ({token.dep_}) - {token.pos_}")

print(f"\nĐường đi: {' -> '.join([t.text for t in path])}")

Path từ 'fox' lên ROOT:
fox (nsubj) - NOUN
jumps (ROOT) - VERB

Đường đi: fox -> jumps

```